Name: Khan Ashhad
Roll Num: 2401048
Batch: T12 [AIDS]

Experiment 9: To provision cloud infrastructure using Terraform

Aim: To provision cloud infrastructure using Terraform

This experiment maps to LO4, LO5, LO6.

LO4:: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

LO5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

LO6: Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.

Theory:

1. Infrastructure as Code

Infrastructure as Code (IaC) is the practice of managing and provisioning computing infrastructure using machine-readable configuration files instead of manually configuring resources through a cloud provider's graphical interface.

IaC makes infrastructure deployment automated, consistent, repeatable, and easier to manage. Terraform is one of the widely used tools for implementing IaC.

In this experiment, Terraform is used to provision an AWS EC2 virtual machine.

2. Terraform

Terraform is an open-source Infrastructure as Code tool developed by HashiCorp. It allows users to define cloud and infrastructure resources using configuration files written in HashiCorp Configuration Language (HCL).

Terraform uses these configuration files to determine:

- Which infrastructure resources are required
- How the resources should be configured
- What changes need to be made

- Which resources should be created, modified, or removed

The Terraform configuration used in this experiment defines an AWS EC2 instance. The lab document identifies Terraform as a method for defining, provisioning, and managing AWS cloud infrastructure through IaC.

3. AWS Provider

Terraform communicates with AWS through the AWS provider.

The provider acts as an interface between Terraform and AWS services. It allows Terraform to create and manage AWS resources such as EC2 instances.

In the experiment, the AWS provider is configured for the Mumbai region (ap-south-1). The Terraform configuration specifies the provider using:

```
provider "aws" {
  region = "ap-south-1"
}
```

The AWS provider is downloaded during the terraform init operation.

4. Terraform Configuration Files

Terraform configuration is written in files with the .tf extension. Multiple .tf files in the same directory are treated as a single Terraform configuration.

The experiment uses:

main.tf

This file defines the Terraform provider and the EC2 instance.

The EC2 resource contains parameters such as:

- AMI ID
- Instance type
- Resource name/tag

The experiment uses an Ubuntu AMI and a t3.micro EC2 instance.

outputs.tf

This file defines the information that Terraform should display after provisioning the resource.

The experiment obtains:

- EC2 instance ID
- Public IP address
- Instance type

The supplied lab document specifically demonstrates these three outputs.

5. Amazon Machine Image (AMI)

An **Amazon Machine Image (AMI)** is a template used to create an EC2 instance. It contains the information required to launch the operating system and associated software environment.

In this experiment, an Ubuntu AMI available in the Mumbai AWS region is used as the operating system image for the EC2 instance.

The AMI must be compatible with the selected AWS region and instance architecture.

6. EC2 Instance

Amazon EC2 (Elastic Compute Cloud) provides virtual servers in the AWS cloud.

In this experiment, Terraform creates an EC2 instance using:

Instance Type: t3.micro

Region: ap-south-1 (Mumbai)

The instance is identified using the tag:

Name = "Terraform-Student-VM"

Terraform manages the EC2 instance instead of requiring it to be manually created through the AWS Console.

7. Terraform Initialization

The first important Terraform command is:

terraform init

It initializes the working directory and downloads the required provider plugins.

For this experiment, Terraform downloads the **HashiCorp AWS provider**.

Successful initialization prepares the directory for further Terraform operations.

8. Terraform Validate

The command:

terraform validate

checks whether the Terraform configuration is syntactically and structurally correct.

If the configuration is valid, Terraform displays:

Success! The configuration is valid.

This allows configuration errors to be detected before infrastructure is created.

9. Terraform Plan

The command:

terraform plan

creates an execution plan showing the changes Terraform intends to make.

For this experiment, the plan showed:

Plan: 1 to add, 0 to change, 0 to destroy.

This means Terraform identified one EC2 resource that needed to be created.

The plan provides an opportunity to review the proposed infrastructure changes before applying them.

10. Terraform Apply

The command:

terraform apply

applies the configuration and creates the resources defined in the Terraform files.

After approval, Terraform created the EC2 instance and displayed information such as:

- Instance ID
- Instance type
- Public IP address

The successful execution produced:

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

11. Terraform Output

The command:

terraform output

displays the output values defined in outputs.tf.

In this experiment, it displayed:

instance_id

instance_type

public_ip

This provides a convenient way to retrieve important information about the provisioned infrastructure.

12. Terraform Destroy

After completing the experiment, infrastructure should be removed using:

terraform destroy

This removes the resources managed by Terraform.

In this experiment, the EC2 instance was successfully deleted, resulting in:

Destroy complete! Resources: 1 destroyed.

The lab instructions specifically require students to perform terraform destroy after the experiment.

Terraform Workflow

The overall workflow followed in this experiment is:

Create Terraform Configuration

↓

terraform init

↓

terraform validate

↓

terraform plan

↓

terraform apply

↓

EC2 Instance Created

↓

terraform output

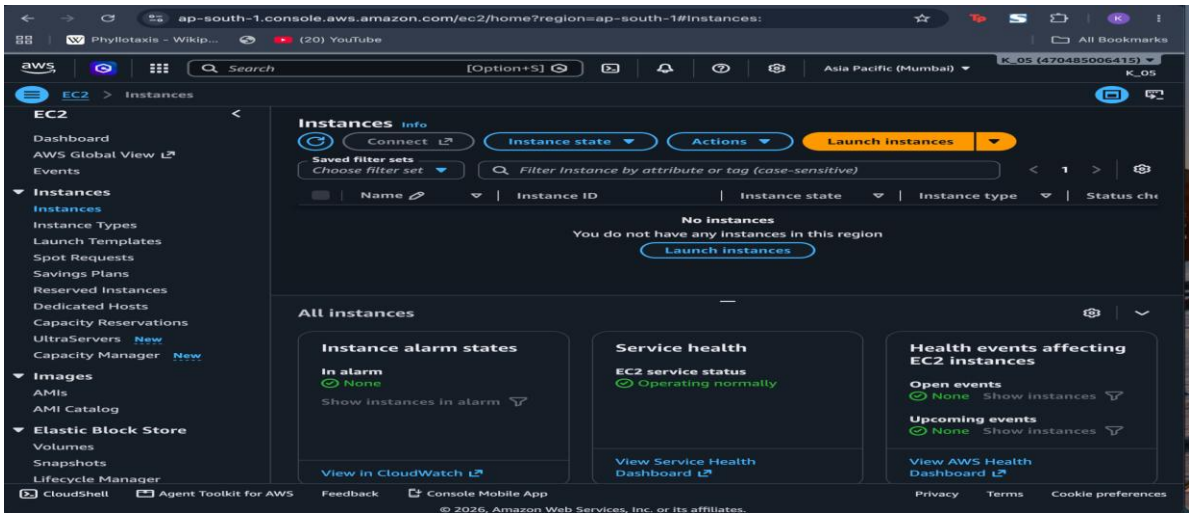
↓

terraform destroy

↓

EC2 Instance Removed

Output:



```
==> Tapping hashicorp/tap
Cloning into '/opt/homebrew/Library/Taps/hashicorp/homebrew-tap'...
Tapped 2 casks and 34 formulae (102 files, 1.8MB).
kyra@Kyras-MacBook-Air ~ % brew install hashicorp/tap/terraform
==> Trusted formula hashicorp/tap/terraform
==> Would install 1 formula:
hashicorp/tap/terraform 1.16.4
==> Fetching downloads for: terraform
✔ Formula terraform (1.16.4)                    Verified     33.9MB/ 33.9MB
==> Installing terraform from hashicorp/tap
🍺  /opt/homebrew/Cellar/terraform/1.16.4: 8 files, 114.6MB, built in 2 seconds
==> Caveats
zsh completions have been installed to:
  /opt/homebrew/share/zsh/site-functions
kyra@Kyras-MacBook-Air ~ % terraform version
Terraform v1.16.4
on darwin_arm64
```

```
Updated profile default to use arn:aws:iam::470485006415:root credentials.

Configure AWS skills and the AWS MCP server for your AI coding agent(s)? [y/n/never]: n
kyra@Kyras-MacBook-Air ~ % aws sts get-caller-identity
{
    "UserId": "470485006415",
    "Account": "470485006415",
    "Arn": "arn:aws:iam::470485006415:root"
}
```

```
kyra@Kyras-MacBook-Air Terraform % grep -n "ami\|instance_type" main.tf
15:  ami = "ami-01a00762f46d584a1"
16:  instance_type = "t3.micro"
kyra@Kyras-MacBook-Air Terraform %
```

```
      + tags_all                                   = {
          + "Name" = "Terraform-Student-VM"
        }
      + tenancy                                    = (known after apply)
      + user_data_base64                           = (known after apply)
      + user_data_replace_on_change                = false
      + vpc_security_group_ids                     = (known after apply)

      + capacity_reservation_specification (known after apply)
```

```
    + tags                                 = {
        + "Name" = "Terraform-Student-VM"
      }
    + tags_all                             = {
        + "Name" = "Terraform-Student-VM"
      }
    + tenancy                              = (known after apply)
    + user_data_base64                     = (known after apply)
    + user_data_replace_on_change          = false
    + vpc_security_group_ids               = (known after apply)

    + capacity_reservation_specification (known after apply)

    + cpu_options (known after apply)
```

```
Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.student_vm: Destroying... [id=i-0de2b5db98a8ce4bf]
aws_instance.student_vm: Still destroying... [id=i-0de2b5db98a8ce4bf, 00m10s ela
psed]
aws_instance.student_vm: Still destroying... [id=i-0de2b5db98a8ce4bf, 00m20s ela
psed]
aws_instance.student_vm: Still destroying... [id=i-0de2b5db98a8ce4bf, 00m30s ela
psed]
aws_instance.student_vm: Destruction complete after 30s

Destroy complete! Resources: 1 destroyed.
```

```
kyra@Kyras-MacBook-Air Terraform % cd ~/Terraform
kyra@Kyras-MacBook-Air Terraform % ls
main.tf         outputs.tf
kyra@Kyras-MacBook-Air Terraform % terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.66.0...
- Installed hashicorp/aws v6.66.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
```

```
Outputs:

instance_id = "i-0de2b5db98a8ce4bf"
instance_type = "t3.micro"
public_ip = "15.252.137.242"
kyra@Kyras-MacBook-Air Terraform % grep -n "ami\|instance_type" main.tf
15:  ami = "ami-01a00762f46d584a1"
16:  instance_type = "t3.micro"
kyra@Kyras-MacBook-Air Terraform % terraform validate
Success! The configuration is valid.

kyra@Kyras-MacBook-Air Terraform % terraform plan
aws_instance.student_vm: Refreshing state... [id=i-0de2b5db98a8ce4bf]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and
found no differences, so no changes are needed.
kyra@Kyras-MacBook-Air Terraform % terraform apply
aws_instance.student_vm: Refreshing state... [id=i-0de2b5db98a8ce4bf]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and
found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0de2b5db98a8ce4bf"
instance_type = "t3.micro"
public_ip = "15.252.137.242"
kyra@Kyras-MacBook-Air Terraform %
```

```
Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_id   = (known after apply)
  + public_ip     = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.student_vm: Creating...
aws_instance.student_vm: Still creating... [00m10s elapsed]
aws_instance.student_vm: Creation complete after 13s [id=i-0de2b5db98a8ce4bf]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0de2b5db98a8ce4bf"
instance_type = "t3.micro"
public_ip = "15.252.137.242"
kyra@Kyras-MacBook-Air Terraform %
```

main.tf outputs.tf

main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "student_vm" {
  ami           = "ami-0018b177b79ff6b47"
  instance_type = "t2.micro"

  tags = {
    Name = "Terraform-Student-VM"
  }
}
```

Conclusion: The experiment successfully demonstrated the use of Terraform for Infrastructure as Code on AWS. We learned how to install and configure Terraform, connect Terraform with AWS using the AWS provider, write Terraform configuration files, and define an EC2 instance as code. We also learned how to use terraform init, terraform validate, terraform plan, terraform apply, and terraform output to initialize, verify, plan, provision, and retrieve information about cloud infrastructure. Finally, we learned how to use terraform destroy to remove the infrastructure after completing the experiment. This experiment provided practical understanding of how IaC can make cloud infrastructure deployment automated, consistent, repeatable, scalable, and efficient, as highlighted in the lab document.

